

Creating Skills on Demand

Presented by Youssef

Can We Create Skills on the Fly?

- What is a Skill?
 - Skills are **reusable**, filesystem-based resources that provide the agent with **domain-specific expertise: workflows, context, and best practices** that transform **general-purpose** agents into **specialists**.
 - Unlike prompts (conversation-level instructions for one-off tasks), Skills load on-demand and eliminate the need to repeatedly provide the same guidance across multiple conversations.
 - Procedure + examples + tool/API knowledge + constraints.
- Key benefits:
 - Specialize Agent: Tailor capabilities for domain-specific tasks
 - Reduce repetition: Create once, use automatically
 - Compose capabilities: Combine Skills to build complex workflows

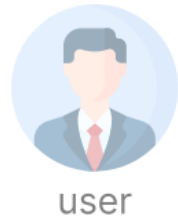
Prior Work

- Most prior work has focused on creating tools on the fly, there is no specific work that has attempted to dynamically generate skills.

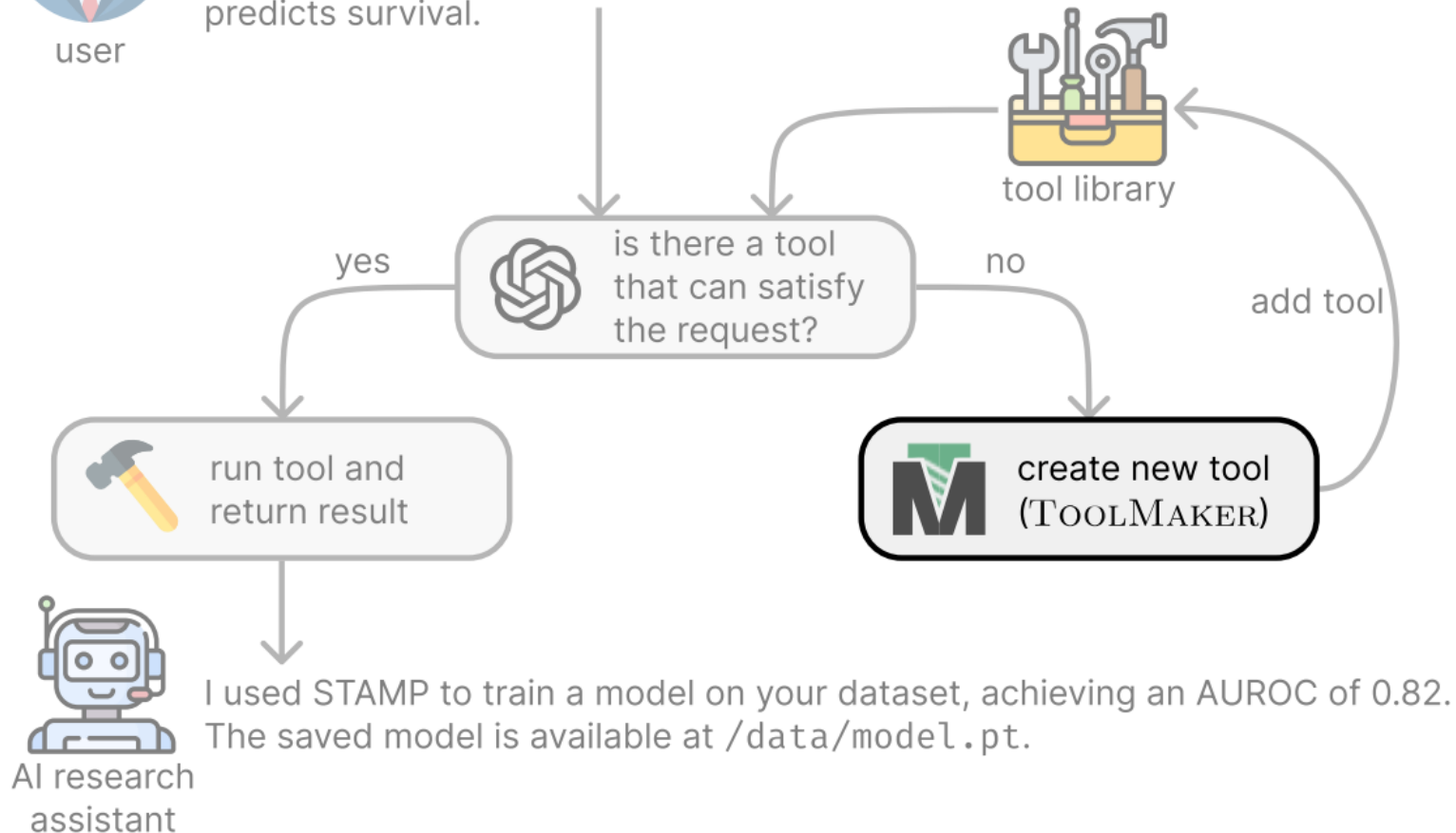
Prior Work: TOOLMAKER

- This paper is about LLM agents that automatically create tools for other LLM agents.
 - Current LLM agents are limited because they can only use tools that humans already implemented and integrated.
 - Useful tools often exist as GitHub repositories attached to papers, but they are hard to install, configure, and call correctly.
 - TOOLMAKER, takes a task description, a scientific paper/code repository, and an example invocation, then produces two things:
 - Docker execution environment
 - Python function implementing the requested tool.

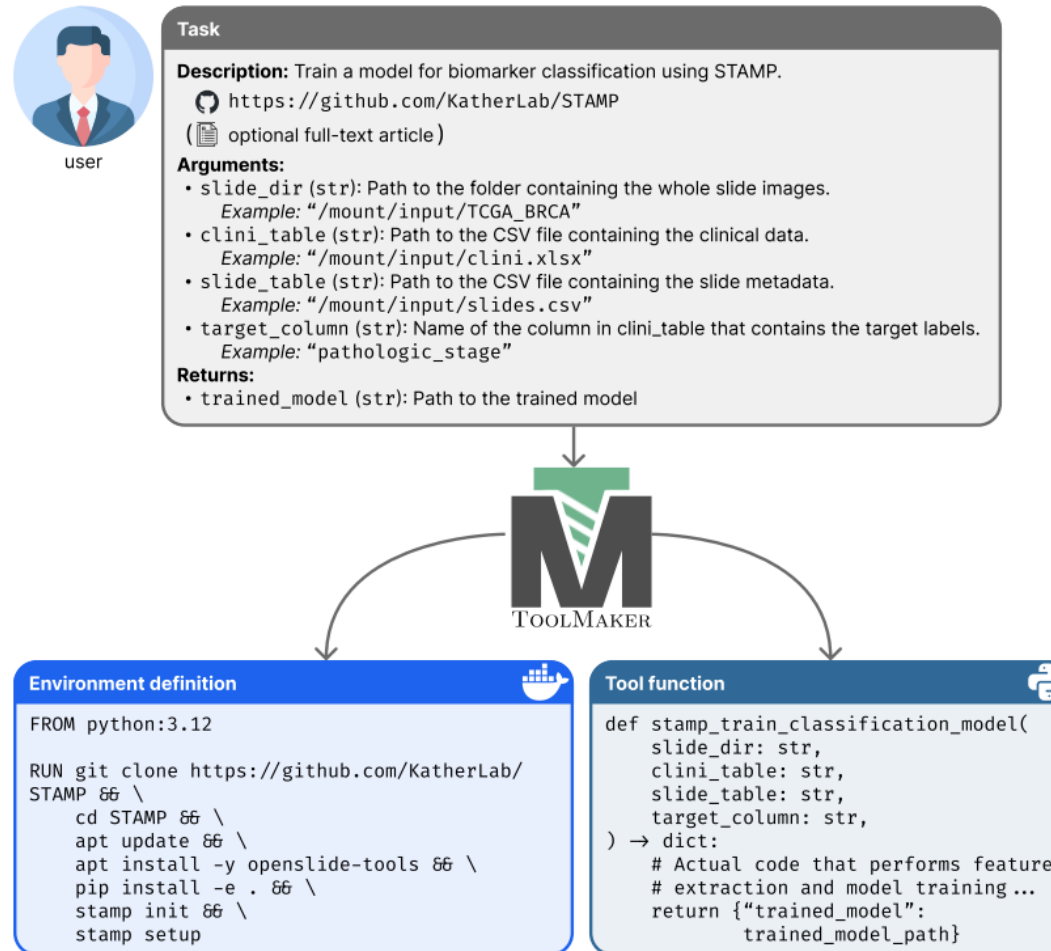
Prior Work: TOOLMAKER



I have a dataset of whole slide images at `/data/images`, and an associated file `/data/labels.csv` containing survival labels for each patient. Train a classification model using STAMP that given an input whole slide image, predicts survival.



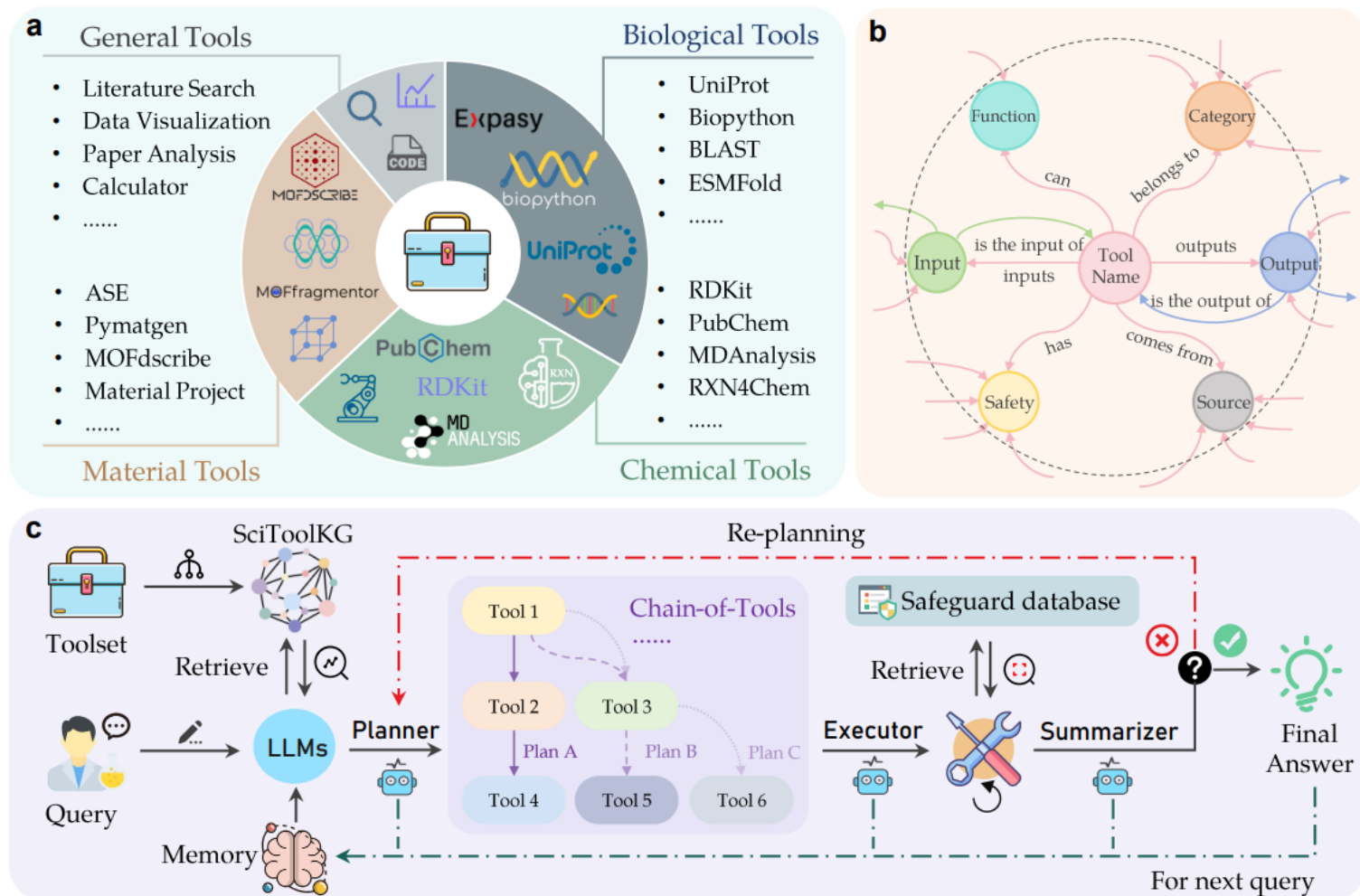
Prior Work: TOOLMAKER



Prior Work: SciToolAgent

- An LLM-based scientific agent that can select and execute many existing scientific tools across biology, chemistry, and materials science.
- The main problem is that scientific tools are powerful but hard to use:
 - They have different input formats, output formats, dependencies, and correct execution orders.
 - A normal LLM agent may know that a tool exists, but it often does not know which tool should come first, which output feeds into which next tool, or whether a result is dangerous.
- SciToolAgent uses a scientific tool knowledge graph to plan, execute, check safety, and summarize multi-tool scientific workflows.

Prior Work: SciToolAgent



Prior Work: SciToolAgent

- Planner:
 - Given a user query q , it does not simply ask the LLM to choose tools from memory. It performs graph-based retrieval over SciToolKG. (In 4 stages)
 - Stage 1: Full-graph retrieval - searches the whole graph for tools that are semantically similar to the user query. $T_{full} = \text{top-}k\{T_i | S(q, T_i), \forall T_i \in G\}$
 - Stage 2: Subgraph exploration - The first retrieved tools may not be enough. So SciToolAgent explores the local neighborhood around each retrieved tool. $T_{sub} = \text{top-}k\{T_j | S(q, T_i \oplus T_j), \forall T_j \in G_{sub, T_i}\}$

Prior Work: SciToolAgent

- Planner:
 - Given a user query q , it does not simply ask the LLM to choose tools from memory. It performs graph-based retrieval over SciToolKG. (In 4 stages)
 - Stage 3: Tool combination and ranking - Then the system ranks tool combinations using: $S' = S(q, T_i) \times S(q, T_i \oplus T_j)$
 - Then it selects the top n tools: $T_{\text{comb}} = \text{top-}n\{T_i | S(q, T_i) \times S(q, T_i \oplus T_j)\}$
 - Stage 4: Chain-of-tools generation - Finally, the LLM receives the selected tools plus graph-neighborhood information and generates an ordered tool chain: $T_{\text{chain}} = \{T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_m\}$

Prior Work: SciToolAgent

- Executor:
 - Input preparation -> Tool execution -> Error handling and retries -> Safety check (It checks risky tool inputs or outputs against a safeguard database.)
- Summerizer:
 - The Summarizer collects all tool outputs and produces the final answer.

Prior Work: CRAFT

- A framework for customizing LLMs by automatically creating a specialized toolset for a task, then retrieving useful tools from that toolset at inference time.
 - Many LLM tool-use systems depend on general-purpose APIs. These are useful, but not always specialized enough. A task may require repeated patterns that are specific to a dataset or domain.
 - Offline phase: create a specialized toolset
 - Online phase: retrieve useful tools and solve a new problem

Prior Work: CRAFT

- Offline phase has 4 stages:
 - **Generation:** Create many correct code solutions from training examples.
 - **Abstraction:** Turns specific code into a reusable function.
 - **Validation:** After abstraction, the tool may become wrong. for example, GPT-4 might over-generalize, remove an important line, or create arguments incorrectly. So CRAFT validates the abstract tool by checking whether it can still solve the original problem.
 - **Deduplication:** Group functions based on (function name, number of input arguments). If multiple tools fall into the same group, GPT-4 is asked to choose the most comprehensive one.

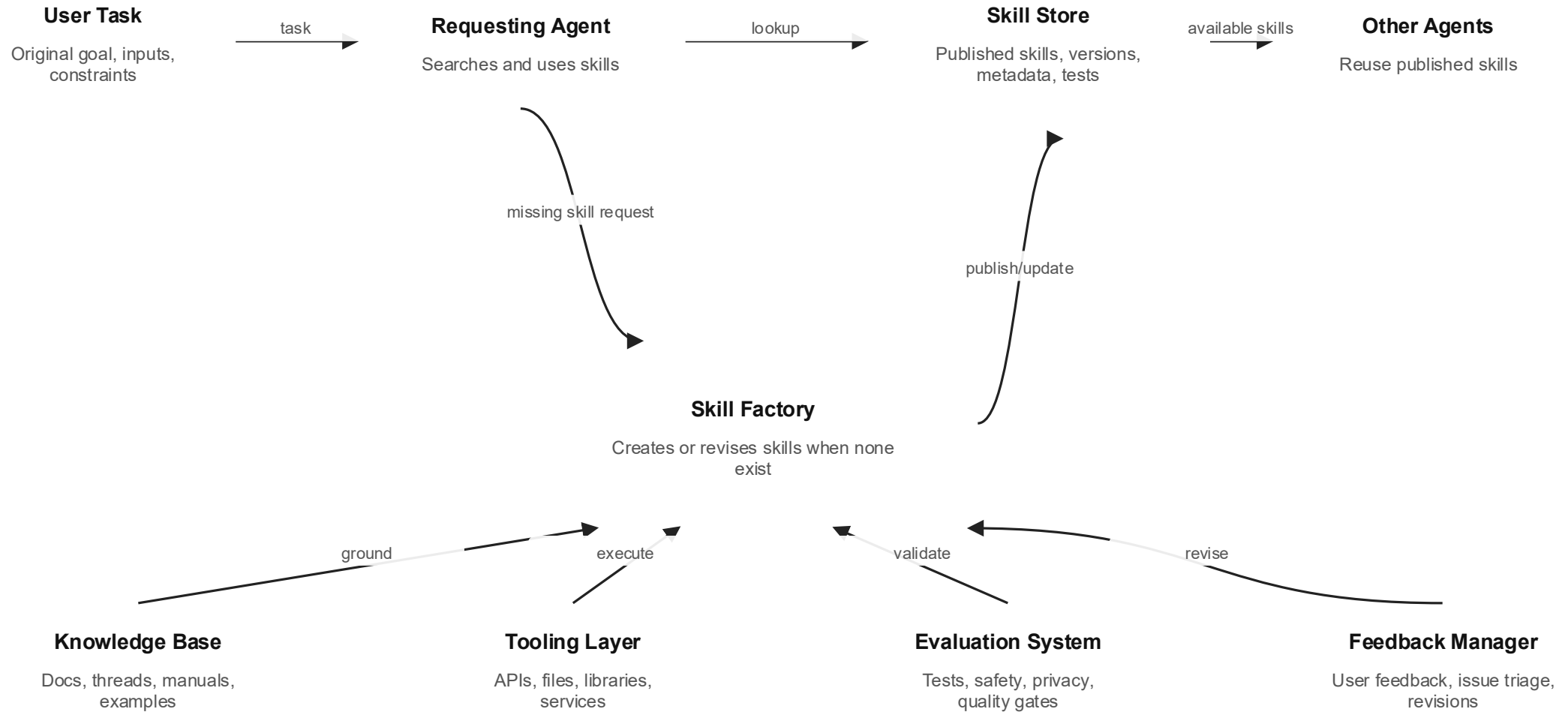
Prior Work: Voyager

- An LLM-powered embodied agent that plays Minecraft as an open-ended lifelong learner.
 - It does not receive a fixed task like “craft a pickaxe” only once. Instead:
 - It continuously decides what to try next
 - Writes executable code to act in the world
 - Learns reusable skills
 - Stores them in a skill library
 - Reuses them later for harder tasks.

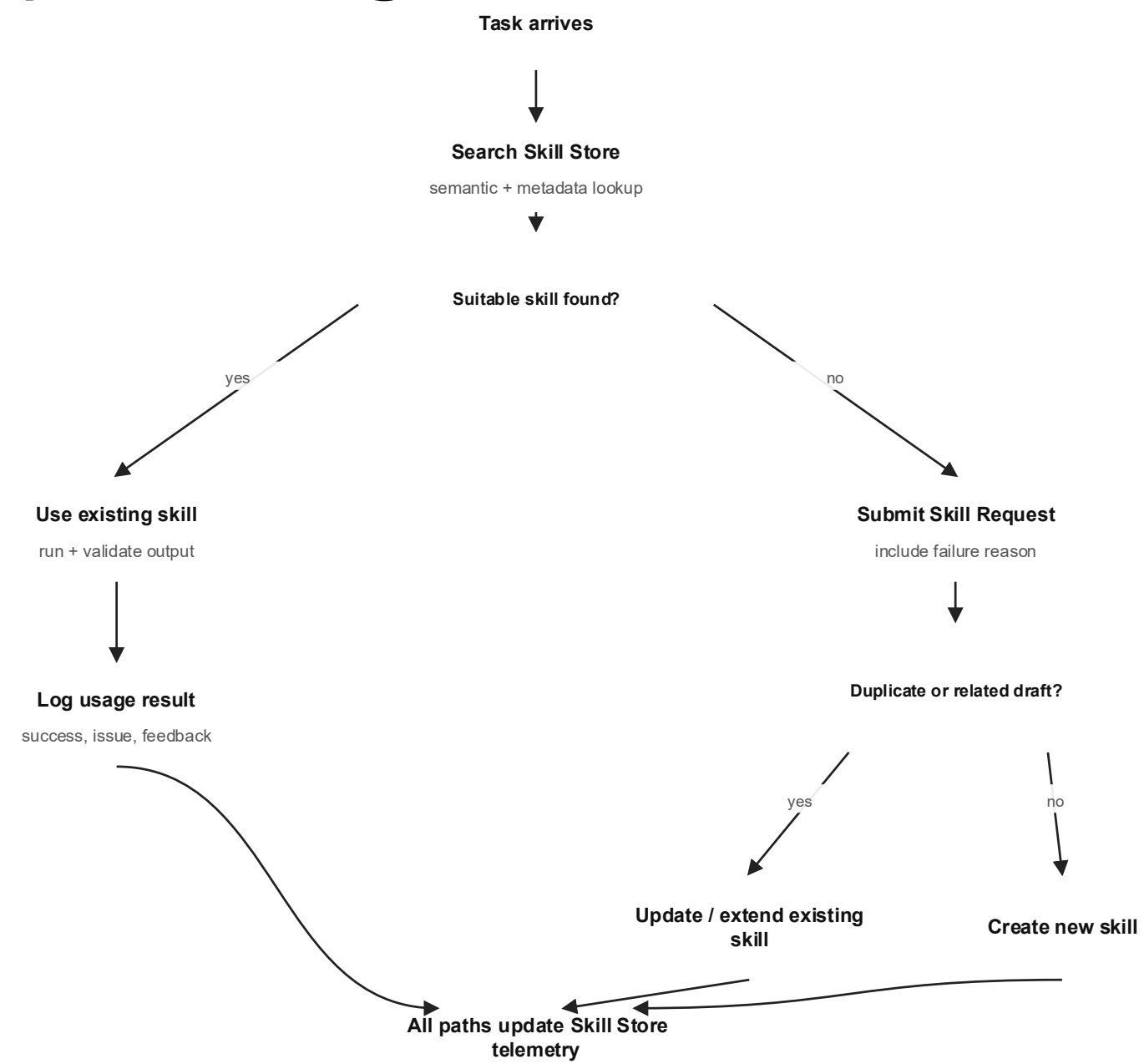
Other Works

- **AutoSkill**: Converts repeated user interactions and preferences into explicit reusable `SKILL.md` skills, then retrieves and injects them in future conversations without retraining the model. (Yang et al., 2026)
- **MUSE-Autoskill**: Treats skills as long-lived assets: agents create, store, manage, test, remember, and refine skills using unit tests, runtime feedback, and skill-level memory. (Lin et al., 2026)
- **SkillX**: Automatically builds a plug-and-play SkillKB from strong-agent trajectories, organizing skills into planning, functional, and atomic levels so weaker agents can reuse them on new tasks. (Wang et al., 2026)

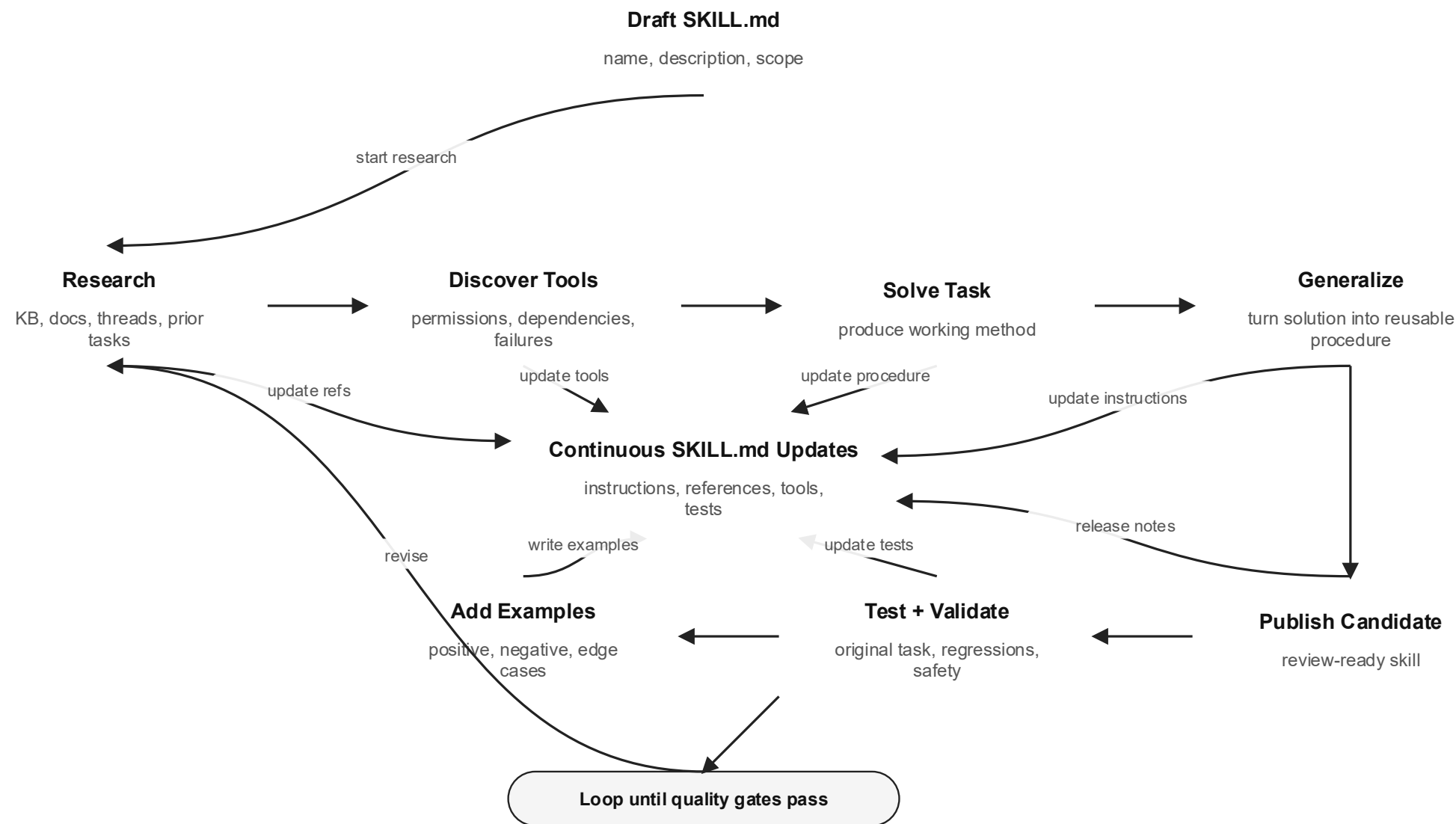
Skill Factory



Skill Factory: Missing Skill Decision Flow



Skill Factory: Skill Factory Inner Loop



Skill Factory: What Gets Updated

Creation Stage		SKILL.md Sections Updated
Intake	→	Metadata · Description · Problem
Triage	→	When to Use · When Not to Use · Related Skills
Initial Definition	→	Inputs · Outputs · Success Criteria · Assumptions
Research	→	References · Background Knowledge · Terminology
Tool Discovery	→	Required Tools · Optional Tools · Dependencies · Permissions
Task Solving	→	Operating Instructions · Decision Tree · Edge Cases
Failure Handling	→	Common Failure Modes · Recovery Steps · Limitations
Instruction Refinement	→	Clarification Policy · Output Format · Validation Checklist
Examples	→	Examples · Negative Examples · Test Fixtures
Testing	→	Evaluation Results · Safety Considerations · Known Limitations
Publication	→	Version · Changelog · Compatibility · Maintainer

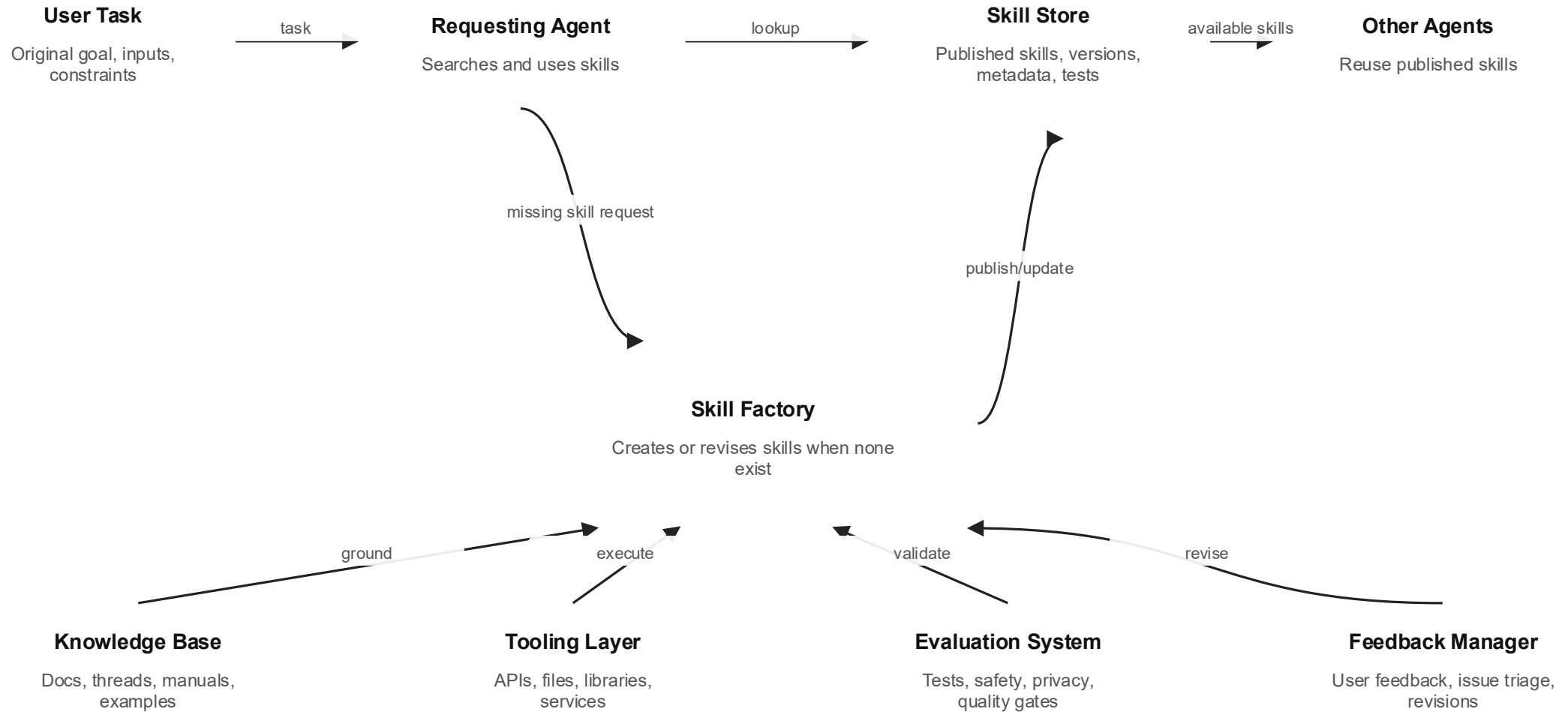
Delta Improvements

- **Cybersecurity KB:** grounds skills in curated security knowledge and operational references.
- **Real-world task generation:** creates skills from actual user/agent needs, beyond benchmark tasks.
- **Full skill lifecycle management:** creates, stores, validates, monitors, updates, versions, and retires skills.

Security Considerations

- Exercise caution when adding scripts to your skill.md file.
- Don't hardcode sensitive information (API keys, passwords).
- Review any skills you download before enabling them.
- Use appropriate MCP connections for external service access.

Skill Factory



System Overview

- 7 stages: Route → Solve → Retrieve → Distill → Validate → Store → Improve
- Core principle: a skill is a generalization, not a transcript of one solve -> That is why we have the two separate stages (because they are very important): solve/**distill** separation and **validation** on held-out instances

Stage 1: The Router (the "else" trigger)

- Implement as orchestration *around* the model.
- On an incoming task: embed its description → vector search existing skill descriptions → plus a model judgment "does any actually apply?"
- Embeddings alone are too blunt (they'll match "edit a PDF" to "edit a Word doc") -> pair with model judgment
- If nothing clears threshold AND model agrees none apply → route to factory
- Expose the factory as a tool / MCP server the model can invoke

Stage 2: The Factory Agent (Solve)

- Shape: a tool-using agent loop (think → call tool → observe → repeat → done)
- Build on the Anthropic Agent SDK (same machinery as Claude Code)
- Run in a bounded sandbox: container with filesystem, code execution, only the tools/network it needs.
- max iterations, max wall-clock, max token budget, max tool calls -> fail loudly, shelve for review
- autonomy works for verifiable tasks (code runs, data validates); fails on underspecified/taste tasks
- Have a "not confident what 'done' means" path that punts to a human queue.

Stage 3: Knowledge Base + Retrieval

- Maybe split the KB by data type:
 - Unstructured prose → chunk + embed → vector store
 - Structured facts/relationships → relational table or graph, queried exactly
 - Prior solve traces + existing skills → their own index
- Hybrid search (BM25 + vector) + a re-ranking step -> pure vector is bad at exact terms (error codes, API/flag names) that skills must get right
- Agentic retrieval: expose the KB as search tools the agent queries iteratively, not one big pre-loaded blob
- Not to make the agent semantically "guess" a fact that it can look up deterministically

Stage 4: Distill

- Distill takes the messy solve trace → produces a generalized, reusable skill
- For every concrete value ask: "instance-specific or part of the general procedure?" → parameterize the former
- Capture: description (when to trigger), generalized procedure, tools, a few examples (≥ 1 input/output pair), real corner cases
- capture corner cases actually encountered, not hypothetical ones (speculative cases may bloat the skill and may often be wrong)

Stage 5: Validation (test generalization, not the instance)

- Once we have the skill from the distillation stage we need to validate it before publishing it.
 - Static checks: valid description, well-formed, references only existing tools, not a near-duplicate (dedup by embedding similarity)
 - Trigger test: paraphrase the task → confirm router selects this skill, and does NOT select it for unrelated tasks (over-triggering is worse than no skill)
 - Held-out execution test: run the agent with the skill on new instances of the same type → check against verifiable success criteria
 - Regression guard: keep the original solve as a permanent test case
- Verifiable tasks → auto-validate; taste-dependent tasks → human sign-off before promotion
- Pass → store; fail → review queue

Stage 6: The Skill Store

- Version every skill -> update from feedback and need rollback when an update regresses
- Store provenance alongside each skill: originating solve + KB docs it drew on (for debugging production issues)
- Dedup on entry so the store doesn't fill with redundant near-copies

Stage 7: Feedback Loop

- Sessions emit structured signals: did it trigger, did the task succeed (same verifiable criteria), where did the agent deviate, did it hit an uncovered corner case
- Don't update per-session -> aggregate; one bad session is noise, a pattern across N sessions is signal
- Guard against feedback poisoning (bad sessions degrading a good skill)
- An update is itself a distill → validate cycle → produces a new version that must clear the same bar (incl. regression guard)
- Diagnose the right fix: consistent failure-to-trigger = description problem; consistent failure-during-execution = procedure problem

The Router

- Detect when no existing skill fits an incoming task → trigger the factory
- Pipeline (4 steps): embed task → vector search skill descriptions → LLM judgment → decide
- Why two stages, not one:
 - Embeddings alone too blunt -> would match "edit PDF" to "edit Word doc"
 - LLM alone doesn't scale -> can't stuff 100 skill descriptions into every prompt
 - Pairing = broad recall (embeddings) + precise semantic gate (LLM)
- Decision rule: low similarity AND LLM says "none apply" → route to factory

The Router

- Embeddings: text-embedding-3-large (3072 dim)
 - Strong retrieval benchmarks (top-tier on MTEB)
 - 3072 dims captures fine semantic distinctions between skill descriptions
 - Standard OpenAI SDK (Available in Azure)
- Judgment: gpt-4o-mini
 - Cheap + fast — runs on every routing decision (hot path)
 - "Does this skill apply?" is a simple call -> flagship intelligence not needed
- Vector store: Qdrant (self-hosted, single Docker container) -> hybrid search (BM25 + vector) built in from day one
- Factory exposed as MCP tool



System Overview

- 7 stages: Route → Solve → Retrieve → Distill → Validate → Store → Improve
- Core principle: a skill is a generalization, not a transcript of one solve -> That is why we have the two separate stages (because they are very important): solve/**distill** separation and **validation** on held-out instances

Prior Work: SkillX - April 21, 2026

- Description
 - SkillX automatically builds a reusable skill knowledge base from agent trajectories.
 - It uses a strong agent to solve tasks, then distills the traces into planning, functional, and atomic skills.
- Limitations
 - Mainly relies on agent exploration/traces, not KBs.
 - Does not provide a full cyber skill lifecycle with domain-specific maintenance and provenance.
 - Depends on a strong agent; if that agent becomes outdated, skill quality may degrade.

Prior Work: SkillsVote - June 14, 2026

- Brief description
 - SkillsVote focuses on skill governance: selecting useful skills before execution and deciding whether traces should update the skill library afterward.
 - It uses subtask-level attribution and only admits successful reusable discoveries into skill updates.
- Limitations
 - It is more about governance/evolution than autonomous domain skill generation.
 - It does not integrate external KBs.
 - Failed or uncertain subtasks do not directly update skills, so it may miss useful failure-driven lessons.

Prior Work: SkillComposer - June 4, 2026

- Brief description
 - SkillComposer trains a model to become good at skill evolution.
 - It decomposes skill construction into create, merge, and improve operations.
- Limitations
 - Skill representation is simple: name, description/trigger, and body.
 - Focus is on training a meta-skill generator, not building a full operational skill lifecycle.

Prior Work: ANYTHING2SKILL - June 19, 2026

- Brief description
 - ANYTHING2SKILL converts external knowledge into reusable procedural skills.
 - It extends RAG by compiling documents, manuals, logs, and trajectories into a persistent SkillBank.
- Limitations
 - Focuses on turning RAG knowledge into procedures, not full task execution.
 - Evaluated on command-line/documentation tasks, not cyber benchmarks.

Delta Improvement

- Cyber-specific skill lifecycle. Cyber KB integration.
- Complete Lifecycle. Continuous maintenance through benchmark scores, failures, new cyber knowledge, and versioning.

Evaluation:

- Our factory ran the full CTIBench CTI-RCM benchmark end-to-end and scored competitively with the published leaders.
- Goal: Given a vulnerability report, identify the correct weakness category behind it.
- The benchmark compares the predicted CWE with the correct CWE using exact-match accuracy.
- Common failure: Models often guess broad categories like CWE-20: Improper Input Validation instead of the more specific correct CWE.

Results

1	alias1 / alias0 (CAIBench)	74%
2	Our factory (gpt-5.4 via Azure)	72.8%
3	DeepSeek-R1-0528	69%
4	qwen3-32B	63%
5	gpt-5 (CAIBench paper)	61%

Knowledge Retrieval:

- A single retrieval method does not work equally well for every query.
- Important limitations:
 - Knowledge may be distributed across several sources.
 - Semantic retrieval may miss exact identifiers.
 - Keyword retrieval may miss paraphrases and related concepts.
 - Retrieved documents may be relevant but still not help solve the task.
 - Searching every source increases latency, cost, and irrelevant context.

R³AG: Selecting the Best Retriever

- R³AG selects one retriever for each query, including a no-retrieval option.
- Candidate retrievers may include BM25, E5, BGE, or other retrieval models.
- It learns two properties:
 - Retrieval quality: Does the retriever find relevant evidence?
 - Generation utility: Does that evidence help the LLM produce the correct answer?
- A query-dependent attention mechanism balances both properties.
- It improves answer quality compared with always using one fixed retriever.
- Limitations:
- Not open source.
- Requires training and labeled retrieval/generation results.
- Selects only one retriever rather than combining several.

RAGRoute: Selecting Knowledge Sources

- RAGRoute is designed for knowledge distributed across multiple repositories.
- It uses a lightweight trained classifier to predict which sources are relevant.
- Only selected sources are searched; their results are merged and reranked.
- It maintains accuracy close to searching every source while reducing latency and communication.
- Limitations:
 - Requires representative source-relevance labels.
 - May require retraining when sources change.
 - Primarily optimizes retrieval efficiency.

Our Project Priorities

- We prioritize:
 - Coverage: avoid missing useful evidence.
 - Accuracy: retrieve authoritative and relevant information.
 - Traceability: preserve the source of every retrieved result.
- We accept slightly higher retrieval cost when it improves task success.
- The agent may search multiple stores rather than selecting only one.
- Our framework is not focused only on knowledge retrieval.
 - May add optimizations afterwards.

Knowledge Sources and Stores

- The knowledge base runs as a separate HTTP service from the agent factory.
- Qdrant prose collection
 - Stores unstructured explanations and document passages.
 - Supports semantic and keyword-based retrieval.
- SQLite structured store
 - Stores exact entities, attributes, types, and relationships.
- Experience index
 - Stores previous tasks, execution traces, successful code, and reusable skills.

Ingestion Pipeline

- Different loaders process different formats:
 - PDF
 - HTML
 - Markdown
 - Text
 - CSV
 - JSON
- Large documents are divided into smaller chunks.
- Metadata is preserved:
 - Document title
 - Page or section
 - Source
 - Version
 - Chunk identifier

Hybrid Retrieval

- The prose store combines dense and sparse retrieval.
- Dense retrieval
 - Uses neural embeddings.
 - Finds passages with similar meaning.
 - Works even when the query and document use different wording.
- Sparse or BM25 retrieval
 - Uses exact words and term importance.
 - Useful for identifiers such as CWE-79 or T1059.003.
- Combining both improves coverage:
 - Dense retrieval captures semantic similarity.
 - Sparse retrieval captures exact technical matches.
- RRF combines results using ranking positions.

LLM Reranking

- Hybrid retrieval first returns a larger group of candidate passages.
- GPT-4.1-mini then scores the top candidates for the specific query.

End-to-End Retrieval

- A thin `search_kb` tool is available to the agent inside the factory container.
- The tool sends the query to the KB service and returns the highest-ranked evidence with source and citation metadata.
- We ran the same CTIBench CTI-RCM benchmark end-to-end task :
 - The agent was able to retrieve the successful trace that we ran earlier and identified from it the common failure we had “models often guess broad categories like CWE-20: Improper Input Validation instead of the more specific correct CWE.”
 - It included explicitly “Prefer specific weakness IDs over generic ones like CWE-20 unless truly necessary.” on its own.
 - Exact match accuracy: 72.8% -> 73.3%

Next Objectives

- Run an ablation study for the different retrieval configuration, to justify our choice:
 - Dense only
 - Dense + sparse
 - Hybrid RRF
 - Structured knowledge
 - LLM reranking
- Include CWE knowledge in our knowledge base and rerun the benchmark task.
- Start with the distillation phase.

Recap

- A thin `search_kb` tool is available to the agent inside the factory container.
- The tool sends the query to the KB service and returns the highest-ranked evidence with source and citation metadata.
- We ran the same CTIBench CTI-RCM benchmark end-to-end task :
 - The agent was able to retrieve the successful trace that we ran earlier and identified from it the common failure we had “models often guess broad categories like CWE-20: Improper Input Validation instead of the more specific correct CWE.”
 - It included explicitly “Prefer specific weakness IDs over generic ones like CWE-20 unless truly necessary.” on its own.
 - Exact match accuracy: 72.8% -> 73.3%

Recap

- Run an ablation study for the different retrieval configuration, to justify our choice:
 - Dense only
 - Dense + sparse
 - Hybrid RRF
 - Structured knowledge
 - LLM reranking
- Include CWE knowledge in our knowledge base and rerun the benchmark task.
- Start with the distillation phase.

CWE Knowledge + Root-Cause Benchmark Rerun

- Added external knowledge
 - Source: MITRE CWE catalog, cwe.mitre.org (XML)
 - Loaded into: fact store (969 CWE entities + 1,160 child-of hierarchy relations) and prose store
- New agent trajectory on the rerun (CTI-RCM, CVE→CWE)
 - Reuses LLM credentials from previous traces (Azure OpenAI, via prior-solve lookup)
 - Per CVE: asks the LLM for a CWE + a self-rated confidence (high / medium / low)
 - On low confidence: fetches the retrieved CWE's neighbors (kb_get + kb_related child-of) and re-prompts the LLM to re-decide among them
 - High-confidence rows kept as-is (no KB call)
- Result: accuracy 73.3% → 74.4%

Retrieval Ablation Study

- Benchmark: MultiHop-RAG (HuggingFace yixuantt/MultiHopRAG)
 - 609 news docs · 2,255 answerable multi-hop queries · relevance labels from evidence
- 5 configs (each adds to the previous) — metrics @10

Config	MRR	MAP	Recall	Hits
Dense only	0.781	0.584	82.2%	98.8%
+ Sparse	0.821	0.635	84.4%	99.4%
Hybrid RRF	0.808	0.617	84.2%	99.4%
+ Structured (metadata)	0.797	0.645	88.2%	99.7%
+ LLM rerank	0.858	0.695	88.2%	99.7%

Distillation Phase: MIND-Skill

- Induction from *successful* traces
- Induction agent: trace → reusable SKILL.md
- Frozen deduction agent re-solves task from skill only (validation)
- 3 signals: reconstruction · outcome · rubric, optimized via TextGrad
- One skill per task;

Distillation Phase: SkillGen

- Contrastive over *success* + *failure* traces
- Compares near-miss failures vs successes → the missing action/check
- Verification gate: deploy only if repairs > regressions
- Needs a volume of failures + held-out set

Distillation Phase

- Our choice: MIND-Skill (Distill)
- Per-trace → matches our data (we produce one trace per run; no failure corpus yet)
- Reuses what we built;
- SkillGen = later (Stage 7 feedback loop). Borrow SkillGen's verification gate as the deploy guard

How We Build Distillation

- Pipeline: distill/
 - Induct (gpt-5.4): trace → SKILL.md — keep general procedure, parameterize specifics, externalize deps (tools, libs, bundled scripts)
 - Deduct (validate): the Factory re-solves from task + skill only, no trace
 - Score: reconstruction similarity · outcome correctness · rubric quality
 - Refine: reflexion loop (Maybe TextGrad later)
 - Gate + Store: keep only if it validates → skills store + router index
- Outcome: new tasks route to use_skill instead of rebuilding from scratch

Our Original Design

Skill Factory — 7 stages



Our Original Design

Skill Factory — 7 stages



Our Plan for Distillation

- Pipeline: distill/
 - Induct (gpt-5.4): trace → SKILL.md — keep general procedure, parameterize specifics, externalize deps (tools, libs, bundled scripts)
 - Deduct (validate): the Factory re-solves from task + skill only, no trace
 - Score: reconstruction similarity · outcome correctness · rubric quality
 - Refine: reflexion loop (Maybe TextGrad later)
 - Gate + Store: keep only if it validates → skills store + router index
- Outcome: new tasks route to use_skill instead of rebuilding from scratch

Our Original Design



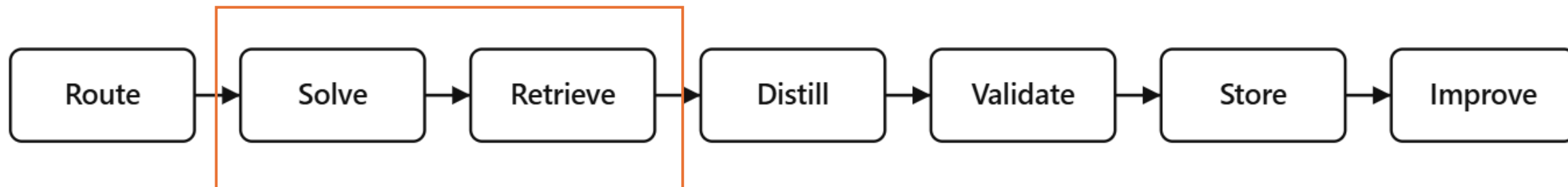
- The generated traces were not always the best performant. The main problem was that not every task has a ground truth to compare against like our benchmark.
- Before using a trace to generate a usable skill. We need to make sure that the approach taken in that trace meets our expectations.
- But what is “our expectations” -> Security engineer success criteria.

Our Original Design



- The given success criteria is given in natural language, and the factory converts it to a checklist that it compares against to evaluate each generated trace.
- If a trace fails -> It is available in our KB with the reason why it failed, and the new agent run would have this available for it to avoid the same problems.

Our Original Design



Engineer's success criteria (free text)

Decide whether the alert is a true or false positive, and justify using specifics from the log. If malicious, map the behavior to MITRE ATT&CK technique IDs (T####) and list the IOCs present in the evidence. Recommend one containment action. Do not over-label without evidence; do not fabricate IOCs not in the input.

normalize
_criteria()

Generated checklist

- ☐ 1 · Decide: true positive vs false positive
- ☐ 2 · Justify with specifics from the log
- ☐ 3 · Map to ATT&CK technique IDs (T####)
- ☐ 4 · List IOCs present in the evidence
- ☐ 5 · Exactly one containment action
- ☐ 6 · Never label malicious without evidence
- ☐ 7 · Never fabricate IOCs not in the input

Our Original Design

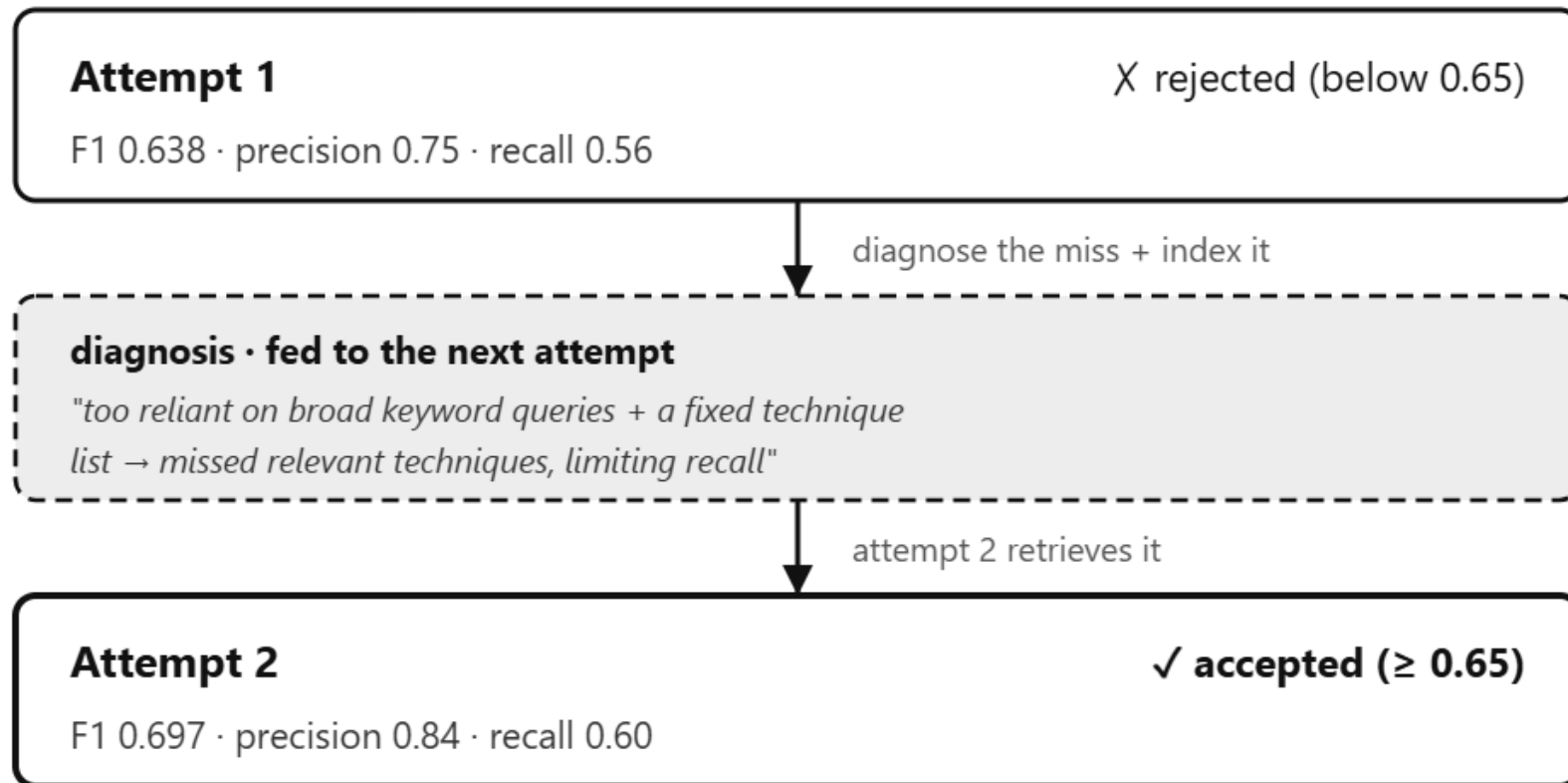


- Distillation → portable skill packages
 - keep general procedure, parameterize specifics, externalize deps (tools, libs, bundled scripts)
 - No system/tool names, the skill carries the knowledge, not instructions to fetch it
- Validation
 - A fresh agent gets only the skill, no access to our tools or knowledge base, and solves examples it has never seen before. If it succeeds, the skill stands on its own and would work on any system.
 - We use TextGrad to improve the skill until it meets the criteria on a standalone agents.

Benchmark: CTI-ATE (ATT&CK Technique Extraction)

- What it is
 - From a malware/threat description, list the MITRE ATT&CK techniques (T####) it shows, multi-label
 - From CTIBench (NeurIPS 2024); 60 real malware descriptions; scored by F1
- Our available KB
 - The MITRE ATT&CK framework (attack.mitre.org), ~700 techniques
- Scores in the paper (zero-shot, no tools)
 - GPT-4: 0.64 (best)

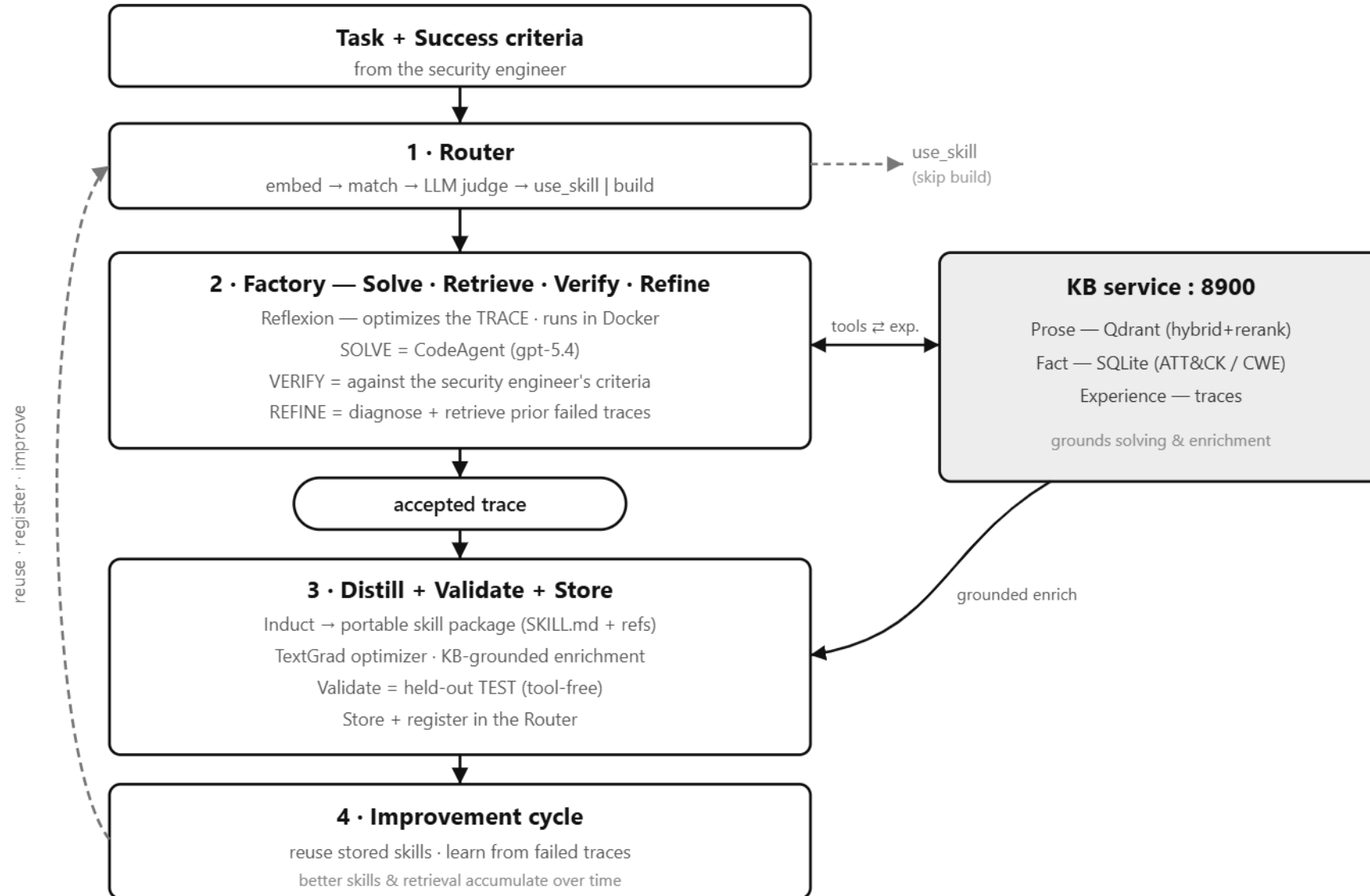
Solve attempts (traces) until one is accepted



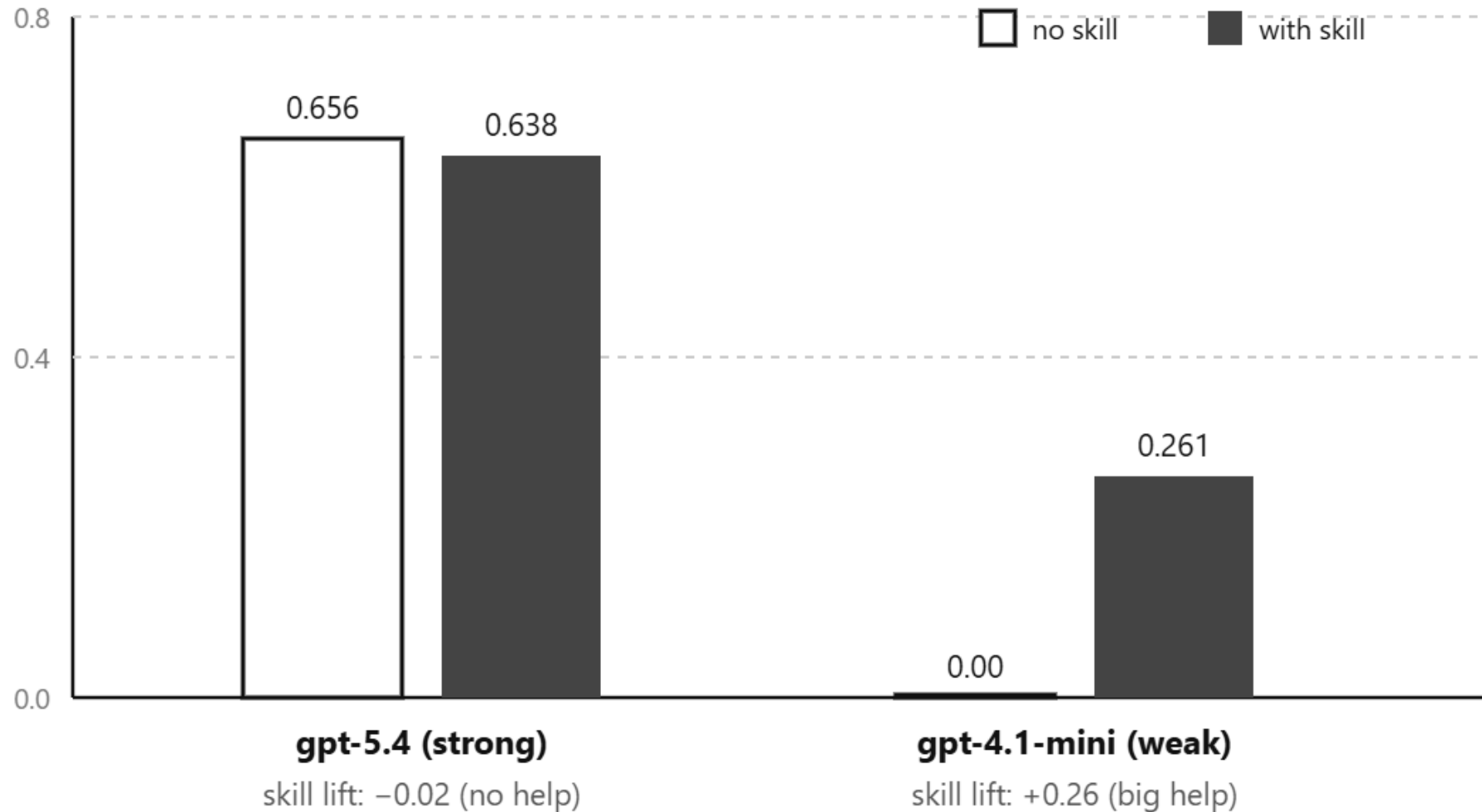
Solve attempts (traces) until one is accepted



Our New Design



Skill Evaluation Experiment



Next Steps

- Investigate the “with-skill” performance:
 - Did the solver discover a new and better procedure?
 - Was the skill forcing the solver to use unnecessary knowledge?
 - Does gpt-5.4 already have sufficient parametric knowledge, making the difference statistically insignificant? This could explain why the skill helps weaker models more.
- Test the approach on several smaller models.
- Test it on more complex tasks with a larger amount of available knowledge.
- Review academic work on the improvement phase to determine how to complete the full pipeline.